

Use Recursion to get sum of N numbers:

1. Non Tail Recursion:

Source Code:

```
#include <iostream>
using namespace std;

int sum(int n){
    if (n==0){
        return 0;
    }
    else if (n==1){
        return 1;
    }
    else{
        return n+sum(n-1);
    }
}

int main(){
    int N;
    cout << "Enter your number: ";
    cin >> N;

    cout << "TotalSum(" << N << ")= " << sum(N)<<endl;
    return 0;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 02_Recursion % gcr 03.1Sum.cpp
Enter your number: 10
TotalSum(10)= 55
```

Discussions:

- Here we used the standard way of recursively evaluating sum of n numbers where we do (n)+sum(n-1).
 - The output was formatted well.
-

2. Tail Recursion:

Source Code:

```
#include <iostream>
using namespace std;

int sum(int n, int result=0){
    if (n==0){
        return result;
    }
    else if (n==1){
        return result+1;
    }
    else{
        return sum(n-1, n+result);
    }
}

int main(){
    int N;
    cout << "Enter your number: ";
    cin >> N;

    cout << "TotalSum(" << N << ")= " << sum(N)<<endl;
    return 0;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 02_Recursion % gcr 03.2Sum.cpp
Enter your number: 12
TotalSum(12)= 78
```

Discussions:

- Here we created a tailed recursion function with default argument sum=1 and updated the sum in each fn call.
 - The output was formatted well.
-

Use Recursion to solve TOH for n rings:

Source Code:

```
#include <iostream>
using namespace std;

void TOH(int count, char src='A', char dest='C', char aux='B'){
    if (count==1){
        cout << "Move from "<< src << " to " << dest << "." <<endl;
    }
    else{
        TOH(count-1, src, aux, dest);
        TOH(1, src, dest, aux);
        TOH(count-1, aux, dest, src);
    }
}

int main(){
    int N;
    cout << "Enter your ring count: "; cin >> N;
    cout << "Perform following moves:" << endl; TOH(N);
    cout << "TOH Solved!" << endl;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 02_Recursion % gcr 04TOH.cpp
Enter your ring count: 4
Perform following moves:
Move from A to B.
Move from A to C.
Move from B to C.
Move from A to B.
Move from C to A.
Move from C to B.
Move from A to B.
Move from A to C.
Move from B to C.
Move from B to A.
Move from C to A.
Move from B to C.
Move from A to B.
Move from A to C.
Move from B to C.
TOH Solved!
```

Discussions:

- The standard algorithm for solving Tower of Hanoi was implemented.
- Here we have 3 recursive call to the function creating a tree recursion structure.
- The moves were printed with readable format.
- The number of moves is given by $(2^n - 1)$.